Derivatives of a Language with Respect to a *Letter*

A useful concept in automating lexers is *derivative* of a language.

Let $L \subseteq A^*$ be a language and let $x \in A$ be a letter. Then

$$\partial_x L = \{w \in A^* \mid xw \in L\}$$

Example: if $L = \{\varepsilon, b, aa, abb, bbba\}$ then

$$\partial_a L = \{a, bb\}$$

$$\partial_b L = \{\varepsilon, bba\}$$

Observe that ∂_x ignores all words that do not begin by x and collects the suffixes of those that begin with x.

If $A = \{a, b\}$ and $L \subseteq A^*$, we can decompose L into disjoint sets:

$$L = (a\,\partial_a L) \cup (b\,\partial_b L) \cup \{\varepsilon \mid nullable(L)\}$$

Derivatives ignore ε if it is in L; the last part puts it back iff L was nullable.

Derivatives of a Language with Respect to a *Word*

Let us generalize the definition from a letter to deriving with respect to an arbitrary word (sequence of letters).

Let $L \subseteq A^*$ be a language and let $v \in A$ be a *word*. Define:

$$\partial_v L = \{w \in A^* \mid vw \in L\}$$

By definition,

$$\boxed{w \in \partial_v L \iff vw \in L}$$

$v' = av$

Because $w \in \partial_\varepsilon L$ means $\varepsilon w \in L$, we have $\partial_\varepsilon L = L$. Moreover,

$$w \in \partial_{av} L \iff avw \in L \iff vw \in \partial_a L \iff w \in \partial_v(\partial_a L)$$

L'

Therefore, $\partial_{av} L = \partial_v(\partial_a L)$, which gives a recursive definition of derivative with respect to a word.

Rules for Derivatives of Regular Expressions

From the meaning of regular expression and the definition of derivative with respect to a letter $a \in A$, we obtain the following derivative rules for regular expressions (remember that $+$ denotes union):

$$\partial_a \varepsilon = \emptyset$$

$$\partial_a \emptyset = \emptyset$$

$$\partial_a b = \begin{cases} \varepsilon & \text{if } b = a \\ \emptyset & \text{if } b \in A \setminus \{a\} \end{cases}$$

$b \in A$ $\quad b \quad \{b\} \xrightarrow{\partial b} \{\varepsilon\}$

$$\partial_a (r_1 + r_2) = (\partial_a r_1) + (\partial_a r_2)$$

$$\partial_a (r_1 \; r_2) = \begin{cases} (\partial_a r_1) \; r_2) + \partial_a r_2 & \text{if } nullable(r_1) \\ (\partial_a r_1) \; r_2 & \text{otherwise} \end{cases}$$

$$\partial_a (r^*) = (\partial_a r) \; r^*$$

Derivative of a regular expression is a regular expression!

Checking if Word Belongs to a Regular Expression

$$w \in L \iff w\varepsilon \in L \iff \boxed{\varepsilon \in \partial_w L} \iff nullable(\partial_w L)$$

If $w = w_1 \ldots w_n$ where $w_1, \ldots, w_n \in A$, then

$$w_1 \ldots w_n \in L \iff \partial_{w_1 \ldots w_n} L \iff nullable(\partial_{w_n} \ldots \partial_{w_1} L)$$

When L is given by a regular expression, we can check the last condition using an algorithm:

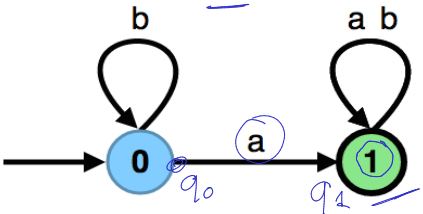
- compute derivative of L with respect to letters of the word, starting from the first letter (at each step we have a regular expression)
- at the end, we check if the final regular expression is nullable (we have seen a recursive algorithm for that in the previous lecture)

However, without further optimizations, this can be inefficient. ZipLex has those optimizations, so it works well. However, for completeness we review a classic approach based on automata.

Finite Automaton (Finite State Machine)

$\Sigma = \{a, b\}$

$A = (\Sigma, Q, q_0, \delta, F)$



$\delta \subseteq Q \times \Sigma \times Q,$

$q_0 \in Q,$

$F \subseteq Q$

$q_0 \in Q$

$q_1 \in Q$

$\delta = \{ (q_0, a, q_1), (q_0, b, q_0), (q_1, a, q_1), (q_1, b, q_1), \}$

- Σ - alphabet
- Q - states (nodes in the graph)
- q_0 - initial state (with '->' sign in drawing)
- δ - transitions (labeled edges in the graph)
- F - final states (double circles)

$abb \in L(A)$

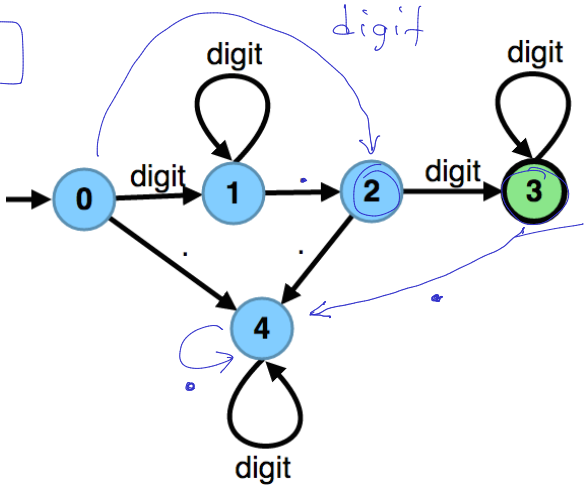
Numbers with Decimal Point

digit digit* . digit digit*

.8

010.11

898.99

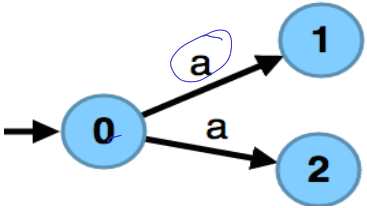


What if the decimal part is optional?

Kinds of Finite State Automata

- DFA: δ is a function : $(Q, \Sigma) \mapsto Q$ (q_0, a, q_2)
- NFA: δ could be a relation (q_0, a, q_1)

- In NFA there is no unique next state. We have a set of possible next states.



Remark: Relations and Functions

- Relation $r \subseteq B \times C$

  $r = \{ \ldots, (b,c1), (b,c2), \ldots \}$

- Corresponding function: $f : B \to 2^C$

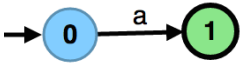
  $f = \{ \ldots (b,\{c1,c2\}) \ldots \}$

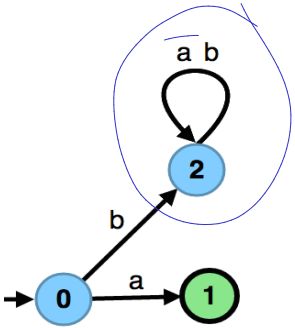
  $f(b) = \{ c \mid (b,c) \in r \}$

- Given a state, next-state function returns a **set** of new states

  for *deterministic* automaton, set has *exactly 1* element

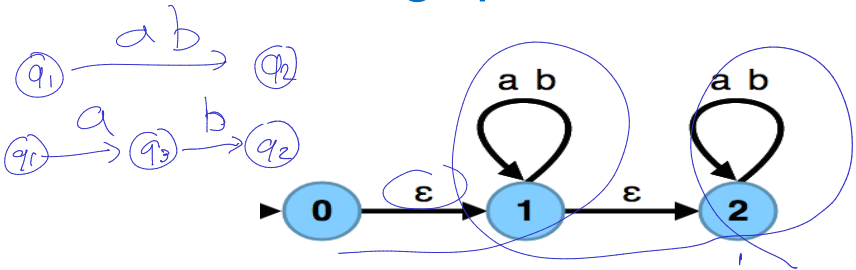
Allowing Undefined Transitions





- Undefined transitions are equivalent to transition into a sink state (from which one cannot recover)

Allowing Epsilon Transitions

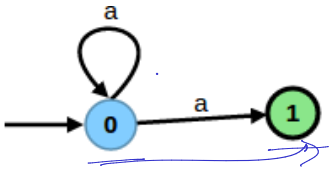


$\varepsilon \cdot a \cdot \varepsilon \cdot b = ab$

- Epsilon transitions:
  - traversing them does not consume anything
- Transitions labeled by a word:
  - traversing them consumes the entire word

When Automaton Accepts a Word

Automaton accepts a word *w* iff there **exists a path** in the automaton from the starting state to some accepting state such that concatenation of words on the path gives *w*.



$1 \in F$

$0 \notin F$

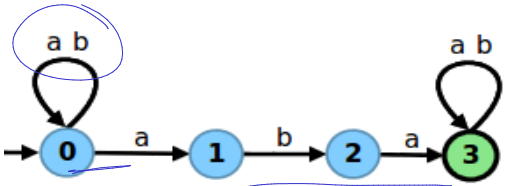
- Does the automaton accept the word ***a*** ?

is there a path from state 0 to 1

Exercise

- Construct a NFA that recognizes all strings over {a,b} that contain "aba" as a substring

only {aba}



$\{0\} \xrightarrow{a} \{0,1\} \xrightarrow{b} \{0,2\} \rightarrow \ldots$

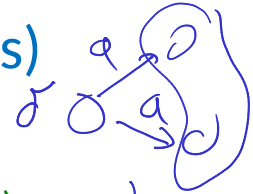
bbabaa

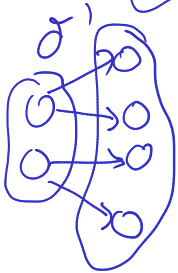
baba

Running NFA (without epsilons)

```
def δ(a : Char)(q : State) : Set[States] = { ... }
def δ'(a : Char, S : Set[States]) : Set[States] = {
  for (q1 <- S, q2 <- δ(a)(q1)) yield q2  // S.flatMap(δ(a))
}

def accepts(input : MyStream[Char]) : Boolean = {
  var S : Set[State] = Set(q0) // current set of states
  while (!input.EOF) {
    val a = input.current
    S = δ'(a,S)          // next set of states
  }
  !(S.intersect(finalStates).isEmpty)
}
```





NFA Vs DFA

- Every DFA is also a NFA (they are a special case)
- For every NFA there exists an equivalent DFA that accepts the same set of strings
- But, NFAs could be exponentially smaller (succinct)
- There are NFAs such that **every** DFA equivalent to it has exponentially more number of states

Regular Expressions and Automata

Theorem:

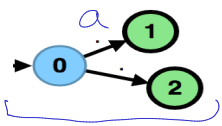
Let L be a language. There exists a regular expression that describes it if and only if there exists a finite automaton that accepts it.

Algorithms:

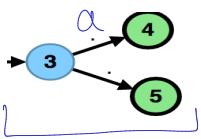
- regular expression → automaton (important!)
- automaton → regular expression (cool)

Recursive Constructions

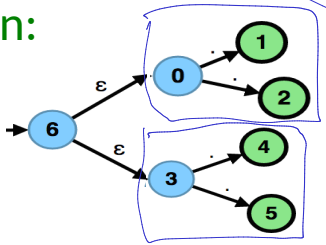
A_1:



A_2:



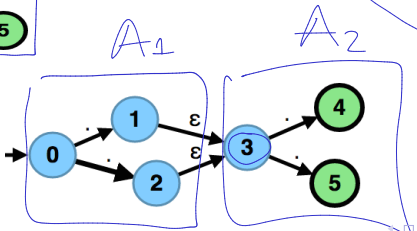
Union:



$r_1 \cup r_2$

$+$

Concatenation:



$r_1 \quad r_2$

$w_1 \quad w_2$

w